



**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Уральский государственный экономический университет»
(УрГЭУ)**

КУРСОВАЯ РАБОТА

по дисциплине **«Объектно-ориентированное программирование»**

на тему: **Разработка обучающей математической игры на Java Swing**

Институт цифровых технологий
управления и информационной
безопасности

Направление подготовки
*09.03.01 Информатика и
вычислительная техника*

Направленность (профиль)
*Программное обеспечение
автоматизированных систем*

Кафедра
*Информационных технологий и
статистики*

Студент

Демшина Анна Александровна

Группа ИВТ-23-1

Руководитель

*Панова Марина Валерьевна,
старший преподаватель, учёный
секретарь кафедры
информационных технологий и
статистики*

Екатеринбург
2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.1 Описание предметной области	5
1.2 Словарь предметной области	6
1.3 Анализ существующих программных решений	8
1.4 Подготовка технического задания	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	13
2.1 Проектирование и разработка классов программного обеспечения	13
2.2 Проектирование пользовательского интерфейса	19
2.3 Описание разработанных алгоритмов и модулей	22
2.4 Описание процесса тестирования программного комплекса	25
2.5 Описание процесса тестирования программного комплекса	27
ЗАКЛЮЧЕНИЕ	30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33

ВВЕДЕНИЕ

В современном мире математическая грамотность критически важна для развития мышления и аналитических способностей, однако традиционные методы обучения часто не обеспечивают достаточной мотивации. В ответ на это, игровые подходы к обучению приобретают особую актуальность, превращая рутинные задачи в увлекательные интерактивные процессы. Обучающие игры, предоставляя немедленную обратную связь и возможность повторения без монотонности, значительно повышают эффективность усвоения знаний. Использование Java Swing позволяет создавать доступные кроссплатформенные приложения с интуитивно понятным графическим интерфейсом. Таким образом, разработка обучающей математической игры актуальна как эффективный инструмент для повышения заинтересованности в изучении математики и развития вычислительных навыков.

Целью данной курсовой работы является разработка и реализация обучающей математической игры с графическим пользовательским интерфейсом на Java Swing, направленной на улучшение навыков устного счета. Для её достижения необходимо решить следующие задачи:

- Изучить принципы построения GUI в Java Swing и основы разработки обучающих игр.
- Спроектировать архитектуру приложения.
- Разработать логику генерации математических примеров различной сложности.
- Реализовать функционал приема и проверки ответов, а также систему подсчета очков.
- Внедрить динамический таймер для ограничения времени решения.
- Предусмотреть возможность выбора уровня сложности.
- Обеспечить интуитивно понятный графический интерфейс.

Объектом исследования в работе выступает процесс разработки интерактивных обучающих приложений, а предметом исследования является программная реализация обучающей математической игры на платформе Java с использованием библиотеки Swing, включая механизмы генерации задач, обработки пользовательского ввода и системы оценки результатов.

Практическая значимость работы заключается в создании функционального программного продукта, который может быть использован в качестве дополнительного образовательного инструмента для самостоятельного изучения и закрепления математических навыков устного счета. Разработанная игра предоставляет интерактивную и адаптивную среду обучения, способствующую повышению мотивации и эффективности учебного процесса.

Научная новизна исследования определяется систематизацией и адаптацией подходов к разработке обучающих игр с динамической сложностью и элементами геймификации в рамках объектно-ориентированной парадигмы на Java Swing. В частности, новизна проявляется в проработке архитектуры, обеспечивающей масштабируемость и легкость расширения функционала для добавления новых типов задач или игровых механик, а также в создании адаптивного алгоритма генерации примеров, учитывающего выбранный уровень сложности.

Таким образом, результатом курсовой работы является разработка интерактивной математической игры, которая может стать эффективным инструментом для улучшения вычислительных навыков и формирования интереса к математике.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

Предметная область данной курсовой работы посвящена интерактивному обучению математике с использованием компьютерных игр. Это междисциплинарное направление, находящееся на пересечении педагогики, информационных технологий и игровой индустрии, ставит своей целью повышение эффективности и привлекательности процесса освоения математических навыков. Основопологающим принципом в данной сфере выступает геймификация, представляющая собой применение игровых элементов и механик в неигровых контекстах для усиления вовлеченности и мотивации пользователей.

В контексте данной предметной области выделяются следующие ключевые термины: обучающая игра, представляющая собой программный продукт, созданный для обучения или развития навыков через игровую деятельность; интерактивность, описывающая динамическое взаимодействие между пользователем и системой; и адаптивное обучение, предполагающее корректировку сложности или содержания материала в соответствии с индивидуальным прогрессом пользователя.

Основными субъектами предметной области выступают пользователи, активно взаимодействующие с обучающей игрой, и разработчики, занимающиеся созданием таких приложений. Объектами являются математические задачи, игровые элементы такие как счет, таймер и уровни сложности, алгоритмы генерации задач, а также графические элементы пользовательского интерфейса.

Взаимодействие субъектов с объектами реализуется посредством ввода пользовательских ответов, выбора уровня сложности, получения немедленной обратной связи от системы и отслеживания собственного прогресса. Использование объектов включает генерацию новых примеров, накопление очков за правильные решения, регулирование времени,

отведенного на задачу, и динамическую адаптацию сложности в соответствии с уровнем подготовки пользователя.

Закономерности, характерные для данной предметной области, включают прямую зависимость эффективности обучения от степени вовлеченности пользователя; критическую важность немедленной обратной связи для закрепления материала; положительное влияние адаптивности на процесс обучения, предотвращающее фрустрацию или потерю интереса; а также стимулирующий эффект соревновательных элементов, побуждающих к улучшению личных результатов.

1.2 СЛОВАРЬ ПРЕДМЕТНОЙ ОБЛАСТИ

Словарь предметной области включает в себя ключевые термины и понятия, используемые для описания и анализа интерактивных обучающих математических игр. Эти слова структурированы по частям речи, что помогает лучше понять функционал и взаимодействие элементов в рамках разрабатываемой системы.

Существительные:

- Игра
- Обучение
- Математика
- Приложение
- Интерфейс (пользовательский, графический)
- Пользователь (учащийся, игрок)
- Разработчик
- Задача (пример, уравнение)
- Число
- Операция (сложение, вычитание, умножение, деление)
- Ответ
- Счет (очки)
- Таймер

- Уровень (сложности)
- Результат (прогресс, достижение)
- Логика (игровая, программная)
- Алгоритм
- Система (подсчета, генерации)
- GUI (графический пользовательский интерфейс)
- Java Swing (библиотека)
- Элемент (игровой, графический)
- Взаимодействие
- Обратная связь
- Мотивация
- Навык (вычислительный, устный)

Глаголы:

- Разрабатывать (создавать, проектировать)
- Обучать (развивать)
- Играть (взаимодействовать)
- Решать (вычислять, отвечать)
- Генерировать (создавать примеры)
- Проверять (анализировать ответы)
- Подсчитывать (считать очки)
- Управлять (временем, процессом)
- Адаптировать (подстраивать сложность)
- Отображать (показывать, выводить)
- Вводить (данные)
- Выбирать (уровень)
- Отслеживать (прогресс)
- Мотивировать
- Закреплять (навыки)

1.3 АНАЛИЗ СУЩЕСТВУЮЩИХ ПРОГРАММНЫХ РЕШЕНИЙ

Анализ существующих программных решений и подходов к созданию обучающих математических игр позволяет выявить эффективные практики, а также определить ниши для дальнейшего развития и уникальности собственного проекта. На сегодняшний день рынок предлагает множество приложений, направленных на развитие вычислительных навыков, которые можно классифицировать по различным критериям: целевая аудитория (дошкольники, школьники, взрослые), используемые платформы (мобильные приложения, веб-платформы, десктопные программы) и подходы к геймификации.

Среди наиболее известных подходов к решению задач интерактивного математического обучения выделяются:

Классические тренажеры устного счета: Это базовые приложения, которые генерируют арифметические примеры, ожидают ввода ответа и предоставляют мгновенную обратную связь. Примеры включают различные онлайн-сервисы и простые мобильные приложения. Их преимущество в простоте и направленности на автоматизацию навыков, но часто им не хватает элементов геймификации для удержания внимания.

Игровые платформы с элементами RPG или приключения: Более сложные решения, где решение математических задач интегрировано в сюжетную линию, например, для открытия новых уровней, получения игровых предметов или победы над противниками. Примеры таких игр включают «Prodigy Math Game», «Math Blaster» или «DragonBox». Эти подходы эффективно повышают мотивацию за счет глубокого погружения в игровую среду.

Адаптивные обучающие системы: Некоторые платформы используют алгоритмы искусственного интеллекта для динамической настройки сложности задач в зависимости от успеваемости пользователя, что позволяет индивидуализировать процесс обучения. Примерами могут служить «IXL»

или «Khan Academy», хотя они не всегда являются играми в чистом виде, но используют адаптивные алгоритмы для подбора заданий.

Соревновательные элементы: Включение таблиц лидеров, возможность соревноваться с друзьями или другими игроками значительно повышает вовлеченность и стимулирует к улучшению результатов.

При анализе конкретных программных решений, таких как математические тренажеры для ПК или «Quick Math» для мобильных устройств, прослеживаются общие черты: наличие генератора случайных примеров, таймера для ограничения времени, системы подсчета очков и базовой обратной связи. Однако, зачастую эти решения обладают ограниченным набором операций, жестко фиксированными уровнями сложности без возможности тонкой настройки, или устаревшим графическим интерфейсом. Некоторые приложения не предоставляют возможности отслеживания лучшего результата по разным уровням сложности, что снижает долгосрочную мотивацию.

1.4 ПОДГОТОВКА ТЕХНИЧЕСКОГО ЗАДАНИЯ

Разработка обучающей математической игры начинается с технического задания (ТЗ), которое определяет цели, задачи и требования к системе. ТЗ обеспечивает единое понимание проекта и является ключевым документом для планирования и контроля.

Функциональные требования описывают основные возможности игры:

- Генерация примеров: Создание арифметических примеров (сложение, вычитание, умножение, деление) с числами, зависящими от сложности.
- Ввод и проверка ответа: Приём ответа пользователя и его сравнение с правильным ответом.
- Подсчет и отображение счета: Учет правильных ответов и показ текущего результата.

- Учет лучшего результата: Сохранение и отображение максимального счета для каждого уровня сложности.
- Ограничение по времени: Установка таймера для каждого примера.
- Выбор уровня сложности: Возможность выбора «Легкого», «Среднего» или «Сложного» уровня.
- Запуск и остановка игры: Функции для начала новой и завершения текущей игры.
- Отображение сообщений: Информирование пользователя о ходе игры (правильность/неправильность, время и т.д.).

Диаграмма вариантов использования:

Акторы: Игрок

Варианты использования:

- Выбрать уровень сложности
- Начать новую игру
- Решить пример: включает «Проверить ответ», «Обновить счет», «Генерировать новый пример», «Обновить таймер».
- Просмотреть текущий счет
- Просмотреть лучший результат
- Остановить игру
- Просмотреть оставшееся время

Пример описания варианта использования: «Решить пример»

Название: Решить пример

Акторы: Игрок

Предусловия: Игра активна, пример и таймер отображены.

Основной поток: Игрок вводит ответ и нажимает «Ответить». Система проверяет:

Правильный ответ: Счет увеличивается, новый пример генерируется, таймер перезапускается.

Неправильный ответ: Выводится правильный ответ, игра останавливается, лучший результат обновляется.

Альтернативные потоки:

- Время вышло: Таймер останавливается, выводится правильный ответ, игра завершается, лучший результат обновляется.

- Некорректный ввод: Выводится сообщение об ошибке, таймер продолжает работать.

Постусловия: Игра продолжается (правильный ответ) или останавливается (неправильный ответ/время вышло).

Требования к экранным формам:

Главное окно: Фиксированный размер (700x450), заголовок «Математическая Игра», светлый фон (AliceBlue), центральное расположение элементов.

Элементы управления:

- Problem Label: Крупный шрифт (Arial, 28).
- Answer Field: Для ввода ответа, широкий, крупный шрифт.
- Submit Button: «Ответить», крупный, синий фон, черный текст.
- Score/Timer/Max Score Labels: Читаемый шрифт (Arial, 22).
- New Game/Change Difficulty Buttons: Аналогичны Submit Button.
- Stop Button: «Стоп», красный фон, белый текст.
- Message Label: Для сообщений, курсив, красный цвет для ошибок.

Диалоговое окно сложности: Появляется при запуске/смене уровня, содержит выбор «Легкий», «Средний», «Сложный».

Модель данных описывает основные параметры состояния игры:

- num1, num2: Числа примера.
- operation: Арифметическая операция.
- correctAnswer: Правильный ответ.
- score: Текущий счет.

- `maxScore`: Лучший результат.
- `gameRunning`: Флаг активности игры.
- `timeLeft`: Оставшееся время.
- `timeExpired`: Флаг истечения времени.
- `difficulty`: Текущий уровень сложности.
- `maxNumber`: Максимальное число для примера на данном уровне.

Нефункциональные требования определяют качество и характеристики работы системы:

- Производительность: Быстрая генерация примеров, плавное обновление таймера.
- Надежность: Корректная обработка некорректного ввода, отсутствие сбоев.
- Удобство использования: Интуитивный, понятный интерфейс для школьников, удобное взаимодействие (мышь/клавиатура).
- Безопасность: Минимальные требования, так как нет работы с конфиденциальными данными.
- Поддерживаемость: Запуск на любой ОС с JVM.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА КЛАССОВ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Процесс проектирования объектно–ориентированного программного обеспечения является ключевым этапом жизненного цикла разработки системы. Он начинается с детализированного анализа предметной области и идентификации основных сущностей, которые впоследствии будут преобразованы в программные классы. Данный этап включает в себя формирование словаря предметной области, где существительные используются для определения потенциальных классов и их атрибутов, а глаголы – для выявления операций (методов). Для разрабатываемой обучающей математической игры, несмотря на её функциональную специфику, применяется систематизированный подход к проектированию, что обеспечивает чистоту кода, его масштабируемость и простоту сопровождения.

Выявление классов, атрибутов и операций

На основе анализа требований, изложенных в техническом задании, и текущей архитектуры программного кода, были идентифицированы следующие ключевые сущности и их характеристики:

Словарь предметной области:

- Существительные (кандидаты в классы/атрибуты): Игра, пример, число, операция, ответ, счёт, время, уровень сложности, кнопка, поле ввода, метка.
- Глаголы (кандидаты в операции/методы): Генерировать, вводить, проверять, подсчитывать, отображать, запускать, останавливать, выбирать, обновлять.

Выявленные классы и их спецификации:

MathGameGUI:

Назначение: Класс MathGameGUI представляет собой основной компонент системы, ответственный за формирование графического пользовательского интерфейса (GUI) и координацию взаимодействия между всеми функциональными элементами игры. Он является главным окном приложения (javax.swing.JFrame).

Атрибуты (поля):

- private static final int TIME_LIMIT_SECONDS_BASE: Базовое время в секундах для решения одного примера.
- private JLabel problemLabel: Компонент для отображения математического примера.
- private JTextField answerField: Компонент для ввода ответа пользователем.
- private JButton submitButton: Кнопка для подтверждения ответа.
- private JLabel scoreLabel: Компонент для отображения текущего счета игрока.
- private JLabel timerLabel: Компонент для отображения оставшегося времени.
- private JButton newGameButton: Кнопка для начала новой игровой сессии.
- private JButton stopButton: Кнопка для принудительной остановки игры.
- private JButton changeDifficultyButton: Кнопка для изменения уровня сложности.
- private JLabel maxScoreLabel: Компонент для отображения наилучшего результата.
- private JLabel messageLabel: Компонент для вывода системных и информационных сообщений пользователю.

- `private int num1, private int num2`: Целочисленные операнды текущего математического примера.
- `private char operation`: Символьное обозначение арифметической операции («+», «-», «*», «/») текущего примера.
- `private int correctAnswer`: Правильный целочисленный ответ на текущий пример.
- `private int score`: Текущее количество очков, набранных игроком в активной сессии.
- `private int maxScore`: Максимальное количество очков, достигнутое игроком за все время для текущего уровня сложности.
- `private boolean gameRunning`: Флаговая переменная, указывающая на активность игровой сессии.
- `private Timer timer`: Объект `javax.swing.Timer` для управления обратным отсчетом времени.
- `private int timeLeft`: Оставшееся время в секундах до истечения таймера.
- `private boolean timeExpired`: Флаговая переменная, указывающая на истечение времени для ответа.
- `private Random random`: Объект `java.util.Random` для генерации случайных чисел и операций.
- `private String difficulty`: Строковое представление текущего уровня сложности («Легкий», «Средний», «Сложный»).
- `private int maxNumber`: Максимальное значение чисел, используемых в примерах, зависящее от выбранного уровня сложности.

Операции (методы):

- `public MathGameGUI()`: Конструктор класса, отвечающий за инициализацию графического интерфейса и его компонентов.

- `public void actionPerformed(ActionEvent e)`: Обработчик событий, реализующий интерфейс `ActionListener` для реакции на действия пользователя, такие как нажатия кнопок.
- `private void chooseDifficulty()`: Метод для предоставления пользователю выбора уровня сложности посредством диалогового окна.
- `private void setDifficultyParameters()`: Метод, устанавливающий диапазон чисел и набор операций в соответствии с выбранным уровнем сложности.
- `private void generateNewProblem()`: Метод для генерации нового математического примера, его отображения в `problemLabel` и перезапуска таймера.
- `private char getRandomOperation()`: Метод, выбирающий случайную арифметическую операцию с учетом текущего уровня сложности.
- `private int calculateAnswer(int num1, int num2, char operation)`: Метод, вычисляющий правильный ответ для заданных операндов и операции.
- `private void startTimer()`: Метод, запускающий обратный отсчет времени для решения примера.
- `private void stopTimer()`: Метод, останавливающий текущий таймер.
- `private void stopGame()`: Метод, завершающий текущую игровую сессию и отображающий итоговый счет.
- `private void checkAnswer()`: Метод, проверяющий корректность ответа пользователя и обновляющий состояние игры.
- `private void startNewGame()`: Метод, инициализирующий новую игровую сессию, обнуляющий счет и запускающий первый пример.
- `private void updateMaxScore()`: Метод, обновляющий значение лучшего результата, если текущий счет его превосходит.

– `public static void main(String[] args)`: Статическая точка входа в приложение, отвечающая за его запуск в потоке диспетчеризации событий AWT.

Отношения между классами и принципы объектно–ориентированного программирования

Текущая архитектура проекта характеризуется наличием одного основного класса `MathGameGUI`, который инкапсулирует как логику игры, так и ее графическое представление. Такой подход является демонстрацией принципа инкапсуляции, при котором данные (атрибуты) и поведение (методы) объекта объединены в единое целое, а внутренние детали реализации скрыты от внешнего доступа.

Абстракция: Класс `MathGameGUI` реализует принцип абстракции, предоставляя пользователю высокоуровневый и упрощенный интерфейс взаимодействия с игрой, при этом скрывая от него сложности внутренней реализации механизмов генерации примеров, управления таймером и обработки ввода.

Композиция: В коде широко применяется принцип композиции, что проявляется во включении экземпляров стандартных классов Java Swing (`JLabel`, `TextField`, `Button`) и служебных классов (`Timer`, `Random`) в состав `MathGameGUI`. Эти компоненты используются для построения интерфейса и реализации функционала игры.

Полиморфизм: Использование полиморфизма наблюдается в реализации интерфейса `ActionListener` методом `actionPerformed()`. Этот метод позволяет унифицировано обрабатывать события от различных источников (например, нажатия разных кнопок), демонстрируя полиморфное поведение.

Для более сложных и масштабируемых программных систем часто рекомендуется разделение ответственности между классами, например, с использованием архитектурного паттерна `Model–View–Controller` (MVC). В таком случае, отдельный класс (`Model`) отвечал бы за игровую логику, другой (`View`) – за пользовательский интерфейс, а третий (`Controller`) – за

координацию их взаимодействия. В контексте данной курсовой работы, совмещение этих ролей в классе MathGameGUI является приемлемым и достаточным решением, упрощающим структуру проекта без потери функциональности.

Спецификации базовых классов и UML–диаграммы

Наименование классов (MathGameGUI), атрибутов (problemLabel, score, difficulty) и методов (generateNewProblem, checkAnswer, startTimer) выполнено в соответствии с общепринятыми соглашениями именования в языке Java (PascalCase для классов, camelCase для полей и методов), что обеспечивает высокую читаемость и понимание кода. Модификаторы доступа (private, public, static) выбраны исходя из принципов инкапсуляции и определения видимости элементов:

- private: Применяется к большинству полей и внутренним вспомогательным методам. Это обеспечивает строгую инкапсуляцию, скрывая детали реализации и предотвращая прямое неконтролируемое изменение извне.

- public: Используется для конструктора класса MathGameGUI() и метода main(), что обеспечивает их доступность для создания экземпляра класса и запуска приложения. Метод actionPerformed() также является public, поскольку это требование интерфейса ActionListener.

- static: Применяется для константы TIME_LIMIT_SECONDS_BASE и метода main(), что означает их принадлежность классу в целом, а не конкретному экземпляру объекта.

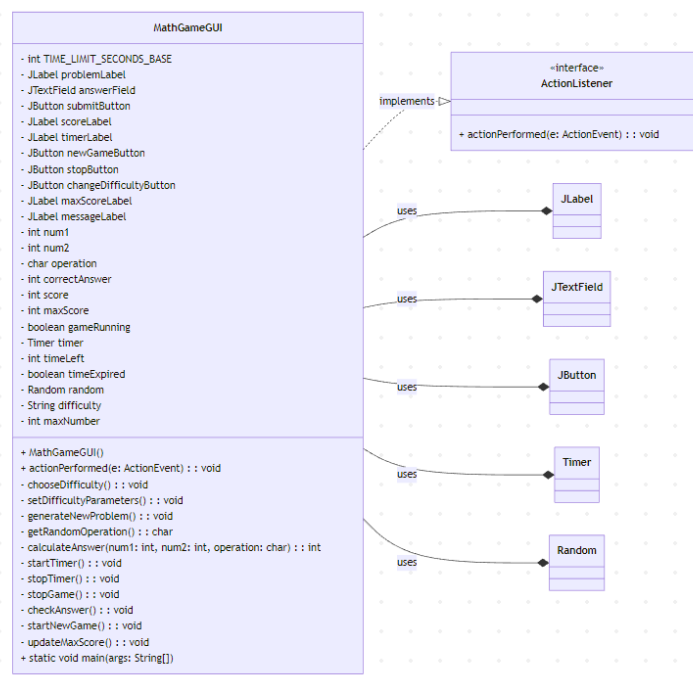


Рисунок 1 – Диаграмма классов

2.2 ПРОЕКТИРОВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Эффективный пользовательский интерфейс (UI) является критически важным компонентом любого программного обеспечения, особенно для обучающих приложений, где интуитивность и удобство взаимодействия напрямую влияют на вовлеченность и результаты пользователя. На данном этапе проектирования детально прорабатывается структура и внешний вид всех элементов управления, их взаимосвязи, а также содержание системных сообщений. Целью является создание интерфейса, который будет максимально понятным, функциональным и приятным для.

Структурная схема пользовательского интерфейса.

Структурная схема пользовательского интерфейса представляет собой высокоуровневое графическое отображение основных экранов (форм) приложения и переходов между ними. Она позволяет визуализировать логику навигации, взаимосвязи между различными частями программы и определить основные точки взаимодействия пользователя с системой. Для обучающей математической игры разработана следующая структурная схема, включающая одно основное окно и одно диалоговое окно:

Диалоговое окно выбора уровня сложности (Difficulty Selection Dialog):

Назначение: Первое окно, которое видит пользователь при запуске приложения или при желании сменить уровень сложности.

Элементы:

- Заголовок: «Выбор уровня»
- Текст-подсказка: «Выберите уровень сложности:»
- Выпадающий список (или радиокнопки): «Легкий», «Средний», «Сложный».
- Кнопка «ОК»
- Кнопка «Отмена»

Переходы:

После выбора уровня и нажатия «ОК» (или по умолчанию при запуске) осуществляется переход к Основному окну игры.

Нажатие «Отмена» (при первом запуске) или закрытие окна прекращает работу приложения.

Основное окно игры:

Назначение: Главный экран, где происходит игровой процесс.

Элементы:

Панель информации и примера:

- Problem Label: Отображает текущий математический пример (например, « $12 + 5 = ?$ »).
- Timer Label: Отображает оставшееся время на решение (например, «Время: 10»).

Панель ввода ответа и управления:

- Answer Field: Поле для ввода числового ответа.
- Submit Button: Кнопка «Ответить» для отправки ответа.

Панель результатов и сообщений:

- Score Label: Отображает текущий счет (например, «Счет: 5»).

- Max Score Label: Отображает лучший результат для текущего уровня сложности (например, «Лучший результат: 12»).
- Message Label: Область для системных сообщений (например, «Правильно!», «Неправильно!», «Время вышло!»).

Панель управления игрой (кнопки):

- New Game Button: Кнопка «Начать новую игру».
- Change Difficulty Button: Кнопка «Сменить уровень» (возвращает к диалоговому окну выбора сложности).
- Stop Button: Кнопка «Стоп» для завершения текущей игры.

Переходы:

- New Game Button: Перезапускает игровую сессию в текущем окне.
- Change Difficulty Button: Возвращает к Диалоговому окну выбора уровня сложности.
- Stop Button: Останавливает игру, остается в этом же окне, но элементы управления для ответа становятся неактивными.

Экраны программного обеспечения

На основе разработанной структурной схемы и пользовательских историй была выполнена реализация графического пользовательского интерфейса. Ниже представлены скриншоты основных экранов программы, демонстрирующие внешний вид и расположение элементов управления в соответствии с дизайн – концепцией и требованиями технического задания.

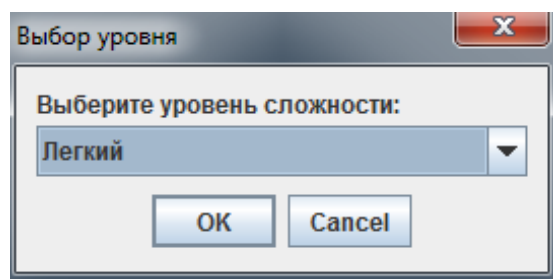


Рисунок 2 – Диалоговое окно выбора уровня сложности

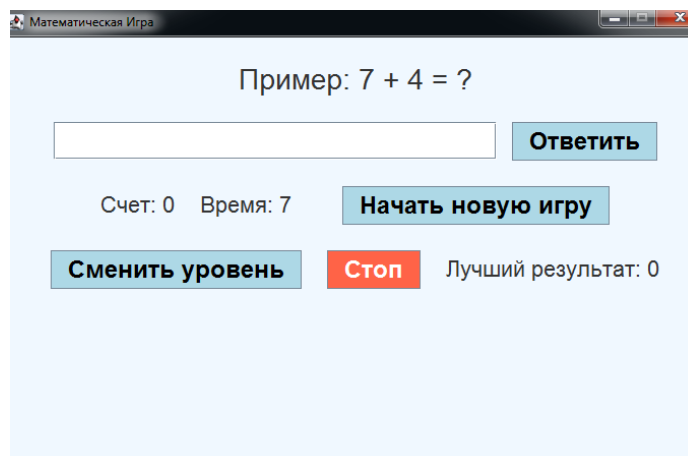


Рисунок 3 – Основное окно игры (активное состояние)

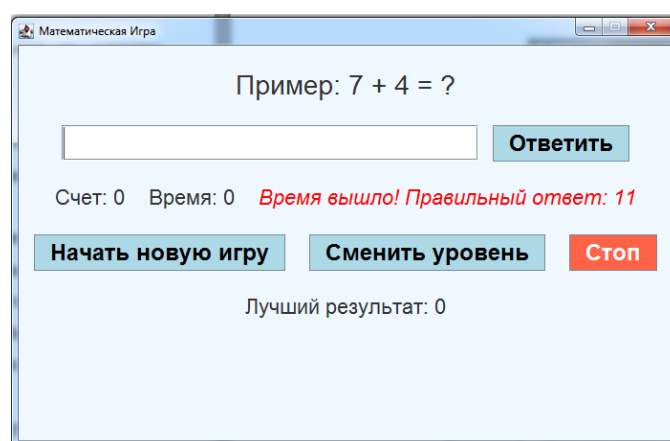


Рисунок 4 – Основное окно игры (состояние «Время вышло»)

Представленные скриншоты наглядно иллюстрируют реализацию элементов управления и информационных полей, обеспечивая интуитивно понятное взаимодействие пользователя с обучающей игрой. Цветовая палитра и типографика были выбраны с учетом эргономических принципов для создания комфортной и продуктивной учебной среды.

2.3 ОПИСАНИЕ РАЗРАБОТАННЫХ АЛГОРИТМОВ И МОДУЛЕЙ

Программный комплекс «Математическая Игра» реализован как единое приложение, основанное на одном основном модуле (классе) – MathGameGUI. Этот класс интегрирует в себе логику управления пользовательским интерфейсом и ключевые аспекты игровой логики. Иерархическая структура: Класс MathGameGUI расширяет JFrame (для GUI) и реализует ActionListener (для обработки событий). Он использует

(композирует) стандартные компоненты Java Swing (JLabel, JTextField, JButton, Timer) и класс Random. Схема связности модулей: Ввиду монолитности MathGameGUI как центрального модуля, схема связности проста. Все взаимодействие осуществляется внутри MathGameGUI или через него к внешним библиотечным классам.

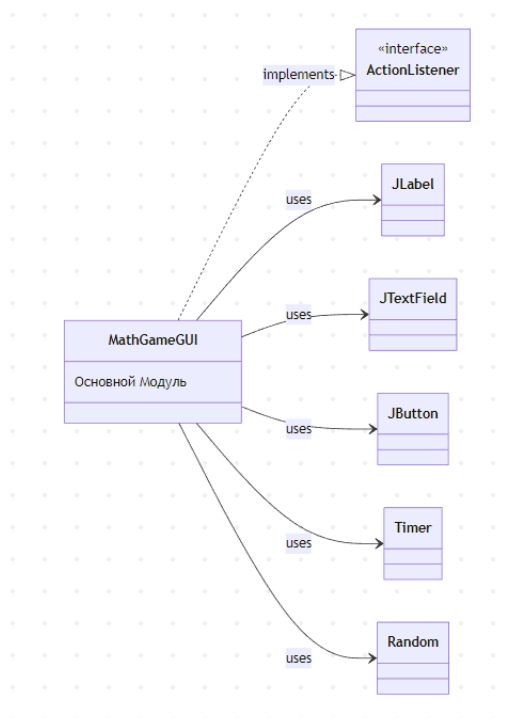


Рисунок 5 – Схема связностей модулей

В программном комплексе реализованы собственные алгоритмы, критически важные для функциональности игры: генерация математических примеров, проверка ответов и управление игровым временем.

Алгоритм генерации нового математического примера (generateNewProblem()):

Отвечает за создание арифметических задач с учетом уровня сложности.

Логика:

- Генерируются два числа (num1, num2) в заданном диапазоне.

- Определяется операция (+, -, *, /) согласно уровню сложности. Для деления на «Сложном» уровне num1 корректируется для обеспечения целочисленного результата.

- Вычисляется correctAnswer.

- Интерфейс обновляется: пример отображается, поле ответа очищается, таймер сбрасывается и запускается.

Алгоритм проверки ответа пользователя (checkAnswer()): Обработывает введенный пользователем ответ, сравнивает с правильным ответом и обновляет состояние игры.

Логика:

- Таймер останавливается.
- Проверяется, не истекло ли время. Если да, игра завершается.
- Введенный текст преобразуется в число.

Если ввод корректен:

При совпадении с correctAnswer: score увеличивается, генерируется новый пример.

При несовпадении: игра останавливается, отображается правильный ответ.

- Если ввод некорректен (не число): выводится сообщение об ошибке, таймер перезапускается.

- В любом случае, после завершения игровой сессии, обновляется maxScore.

Алгоритм управления игровым таймером (startTimer() и stopTimer()): Обеспечивает обратный отсчет времени для каждого примера и реакцию на его истечение.

Логика:

- startTimer(): останавливает предыдущий таймер, создает новый Timer с интервалом 1 секунда.

- Каждую секунду: уменьшает timeLeft, обновляет timerLabel.
- При timeLeft ≤ 0 : таймер останавливается, игра завершается, отображается сообщение о времени и обновляется maxScore.
- stopTimer(): прерывает работу активного таймера.

Эти алгоритмы составляют ядро функциональной логики обучающей математической игры, обеспечивая её интерактивность и соответствие требованиям к игровому процессу.

2.4 ОПИСАНИЕ ПРОЦЕССА ТЕСТИРОВАНИЯ ПРОГРАММНОГО КОМПЛЕКСА

Тестирование проводилось методом «черного ящика», сфокусированным на функциональности программы. Были разработаны и выполнены следующие основные тестовые сценарии:

- Тестирование выбора уровня сложности: Проверка корректности установки параметров игры и диапазона чисел для «Легкого», «Среднего» и «Сложного» уровней, включая деление.
- Тестирование генерации примеров: Проверка разнообразия и корректности генерируемых примеров, включая отсутствие деления на ноль и целочисленные результаты.
- Тестирование ввода и проверки ответа: Проверка обработки правильных, неправильных, нечисловых и пустых ответов пользователя.
- Тестирование таймера: Проверка корректности обратного отсчета, реакции на истечение времени и влияния на игровой процесс.
- Тестирование кнопок управления игрой: Проверка функциональности кнопок «Начать новую игру», «Сменить уровень», «Стоп».

Анализ результатов тестирования

Положительные результаты:

- Все заявленные функциональные требования успешно реализованы и протестированы.

- Интерфейс интуитивно понятен и прост в использовании.

- Генерация примеров и работа таймера функционируют корректно и стабильно.

- Обработка пользовательского ввода осуществляется правильно.

Выявленные недостатки (потенциальные улучшения):

- Отсутствие сохранения результатов между сессиями.

- Необходимость ручной очистки поля ввода после некорректного ввода.

Достоинства и недостатки разработки

Достоинства разработки:

- Простота и наглядность: Чистая и понятная структура кода.

- Высокая функциональность: Реализован весь базовый функционал игры.

- Инкапсуляция: Принципы инкапсуляции применены, что способствует модульности.

- Кроссплатформенность: Приложение работает на различных ОС благодаря Java Swing.

Недостатки разработки:

- Монолитная архитектура: Концентрация всей логики в одном классе может затруднить масштабирование и добавление сложных функций.

- Отсутствие модульного тестирования: Акцент на функциональном тестировании без детализированных юнит-тестов.

- Ограниченное использование паттернов: Применение паттернов (например, MVC) могло бы улучшить архитектуру.

В целом, разработанный программный комплекс является функциональным и соответствует поставленным целям, а выявленные

недостатки представляют перспективные направления для дальнейшего совершенствования.

2.5 ОПИСАНИЕ ПРОЦЕССА ТЕСТИРОВАНИЯ ПРОГРАММНОГО КОМПЛЕКСА

Для обеспечения корректного функционирования программного комплекса необходимо, чтобы на компьютере пользователя была установлена среда исполнения Java (Java Runtime Environment, JRE) версии 8 или выше. Приложение не требует дополнительной инсталляции и обладает кроссплатформенной совместимостью, функционируя на различных операционных системах при наличии соответствующей версии JRE.

Процедура запуска программы:

- Перед запуском приложения требуется убедиться в наличии установленной JRE на операционной системе.
- Определение местоположения исполняемого JAR-файла приложения, имеющего наименование MathGame.jar.

Инициирование запуска осуществляется следующими способами:

- Для операционных систем семейства Windows: Производится двойной щелчок левой кнопкой мыши по файлу MathGame.jar.
- Для операционных систем macOS и Linux: Осуществляется открытие терминала, переход в директорию, содержащую файл MathGame.jar, после чего вводится следующая команда: `java -jar MathGame.jar`

Конфигурирование уровня сложности

При первичном запуске программы или после активации функции «Сменить уровень» посредством соответствующей кнопки, отображается диалоговое окно для выбора уровня сложности:

В диалоговом окне с заголовком «Выбор уровня» осуществляется выбор желаемого уровня сложности из представленного списка:

- Легкий: Предусматривает генерацию простых арифметических примеров (операции сложения и вычитания) с использованием чисел в небольшом диапазоне.

- Средний: Включает примеры средней сложности (операции сложения, вычитания и умножения) с использованием чисел в расширенном диапазоне.

- Сложный: Генерирует сложные арифметические примеры (все четыре базовые операции, включая деление) с использованием чисел в наибольшем диапазоне и сокращенным временным лимитом на ответ.

Для подтверждения выбора и перехода к игровому процессу производится нажатие кнопки «ОК».

Для отмены выбора или закрытия диалогового окна без изменения настроек выполняется нажатие кнопки «Отмена» или стандартной кнопки закрытия окна.

Особенности игрового процесса

- После выбора и подтверждения уровня сложности активируется основное окно игры, где разворачивается игровой процесс:

- Отображение примера: В верхней области окна представляется текущий математический пример (например, « $15 + 7 = ?$ »).

- Индикация времени: Справа от поля примера расположен индикатор таймера, отображающий оставшееся время для ввода ответа.

- Ввод ответа: Пользователем вводится числовое значение ответа в соответствующее текстовое поле, расположенное под отображаемым примером.

- Подтверждение ответа: Для валидации введенного ответа осуществляется нажатие кнопки «Ответить» или использование клавиши Enter на клавиатуре.

- Системная обратная связь:

Корректный ответ: В случае верного ответа, текущий счет игрока инкрементируется, и генерируется новый математический пример.

Некорректный ответ: При вводе неверного ответа игровой процесс приостанавливается, и программой отображается правильный ответ.

Истечение времени: В случае, если ответ не был введен до истечения установленного таймером времени, игра также останавливается, и показывается корректный ответ.

– Мониторинг результатов: В нижней части основного окна постоянно отображаются актуальные значения «Текущего счета» и «Лучшего результата», достигнутого для активного уровня сложности.

Управление игровой сессией.

В основном окне игры предусмотрены следующие элементы управления для манипуляции игровой сессией:

– Кнопка «Начать новую игру»: Иницирует новый игровой цикл. При её активации текущий счет обнуляется, таймер перезапускается, и генерируется новый пример, соответствующий текущему уровню сложности.

– Кнопка «Сменить уровень»: Открывает диалоговое окно для выбора другого уровня сложности. После выбора нового уровня игра начнется заново с обновленными параметрами.

– Кнопка «Стоп»: Предназначена для немедленного прекращения текущей игровой сессии.

Процедура завершения работы программы.

Для завершения работы приложения осуществляется закрытие его окна, используя стандартную кнопку «Закрыть», расположенную в правом верхнем углу окна.

ЗАКЛЮЧЕНИЕ

Представленный программный комплекс, разработанный в рамках данного исследования, представляет собой интерактивную систему «Математическая Игра», предназначенную для тренировки арифметических навыков пользователей различных уровней подготовки. В процессе выполнения работы был реализован полный цикл разработки программного обеспечения, охватывающий этапы от анализа требований до финального тестирования и документирования.

Основные результаты и примененные методологии:

- Анализ требований и объектно-ориентированное проектирование: Осуществлена всесторонняя декомпозиция предметной области, результатом которой стала разработка архитектуры на основе класса MathGameGUI. Данный класс эффективно инкапсулирует как логику игрового процесса, так и функциональные элементы графического пользовательского интерфейса. В ходе проектирования активно применялись фундаментальные принципы объектно-ориентированного программирования: инкапсуляция, абстракция, полиморфизм, а также композиция с использованием стандартных компонентов Java Swing. Структура приложения была формализована посредством UML-диаграмм классов.

- Проектирование пользовательского интерфейса: Выполнена разработка структурной схемы пользовательского интерфейса и определены ключевые пользовательские истории, описывающие сценарии взаимодействия с системой. Визуализация реализованного интерфейса представлена посредством скриншотов основных экранов приложения, демонстрирующих интуитивно понятную и функционально оптимальную организацию элементов управления.

- Разработка алгоритмов и программных модулей: Детально описаны и реализованы основные алгоритмы, обеспечивающие базовую

функциональность игрового процесса. К ним относятся алгоритмы генерации математических примеров (с учетом варьирования сложности и типов арифметических операций), алгоритмы валидации ответов пользователя и алгоритмы управления временными параметрами игровой сессии.

– Тестирование и оценка системы: Осуществлено функциональное тестирование программного комплекса с применением метода «черного ящика», которое подтвердило корректность реализации всех специфицированных требований. Выявленные в процессе тестирования потенциальные векторы улучшения и недостатки архитектурного решения (например, монолитная структура и отсутствие персистентного сохранения результатов) были проанализированы для определения направлений дальнейшего развития проекта.

Оценка полноты решения поставленных задач:

Все задачи, сформулированные в техническом задании на разработку обучающей математической игры, были решены в полном объеме. Созданный программный комплекс полностью функционален, обеспечивает генерацию примеров различной степени сложности, корректную обработку пользовательских ответов, ведение счета и управление игровым временем.

Значимость исследования и область применения полученных результатов:

Значимость выполненного исследования заключается в создании эффективного интерактивного инструмента для обучения и самоконтроля в области базовой арифметики. Разработанный программный комплекс находит потенциальное применение в следующих контекстах:

– Образовательные учреждения: В качестве дидактического средства для учащихся начальных и средних классов, способствующего закреплению математических знаний и навыков.

– Самостоятельное обучение: Для индивидуальной тренировки арифметических способностей.

– Развивающие центры: Как компонент интерактивных программ, направленных на развитие логического мышления и скорости вычислительных операций.

Полученные результаты демонстрируют компетенции в области проектирования, разработки и тестирования программного обеспечения на платформе Java, а также понимание принципов построения эффективных пользовательских интерфейсов. Перспективы развития проекта включают расширение функциональности за счет добавления новых возможностей, таких как ведение пользовательской статистики, поддержка многопользовательского режима, более гибкая конфигурация сложности и интеграция сетевых функций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Блох Д. Java. Эффективное программирование: моногр. / Д. Блох. – 3-е изд. – Москва : ДМК Пресс, 2021. – 480 с.

Вагнер Г. Тестирование программного обеспечения: от основ до продвинутых техник: учеб. пособие / Б. Вагнер. – Москва : Вильямс, 2023. – 416 с.

Волкова И. А. Объектно-ориентированное программирование и паттерны проектирования: учебник / А. А. Волков. – Санкт-Петербург : БХВ-Петербург, 2022. – 384 с.

Мартин Р. Чистый Agile: Уроки по гибкой разработке программного обеспечения: моногр. / Р. Мартин. – Москва : Питер, 2020. – 304 с.

Прессман Р. С. Инженерия программного обеспечения: Прикладной подход: учебник / Р. С. Прессман, Б. Р. Максим. – 8-е изд., перераб. и доп. – Санкт-Петербург : Питер, 2021. – 960 с.

Трофимов В. В. Алгоритмы и структуры данных: теория и практика: учеб. пособие / В. В. Трофимов, В. Н. Коптева. – Москва : КНОРУС, 2024. – 288 с.

Шилдт Г. Java. Полное руководство: учебник / Г. Шилдт. – 11-е изд. – Санкт-Петербург : Диалектика, 2021. – 1473 с.

Гамма Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования: моногр. / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург : Питер, 2022. – 368 с.

Макконнелл С. Совершенный код: моногр. / С. Макконнелл. – 2-е изд. – Москва : Русская Редакция, 2021. – 896 с.

Oracle. Использование компонентов Swing (Using Swing Components) [Электронный ресурс] : Учебное пособие. – Обновлено: 2021. – Режим доступа: <https://docs.oracle.com/javase/tutorial/uiswing/index.html> (дата обращения: 07.05.2025)

Веретельников Д. Тестирование программного обеспечения: гайд для начинающих [Электронный ресурс] : Статья // Хабр. – 2023. – Режим доступа: <https://habr.com/ru/companies/sberservices/articles/734268/> (дата обращения: 07.05.2025).

Metanit.com. Java Swing [Электронный ресурс] : Руководство. – Обновлено: 2024. – Режим доступа: <https://metanit.com/java/swing/> (дата обращения: 07.05.2025).

Refactoring.Guru. Паттерны проектирования [Электронный ресурс] : Руководство. – Режим доступа: <https://refactoring.guru/ru/design-patterns> (дата обращения: 07.05.2025)

ПРИЛОЖЕНИЕ А

Листинг 1. Программа MathGUI1

```
import javax.swing.*; // Импорт основных классов для создания
графического интерфейса Swing
import java.awt.*; // Импорт классов AWT для работы с графикой и
пользовательским интерфейсом (например, Font, Color, FlowLayout)
import java.awt.event.ActionEvent; // Импорт класса для обработки
событий действия (например, нажатия кнопки)
import java.awt.event.ActionListener; // Импорт интерфейса для
слушателей событий действия
import java.util.Random; // Импорт класса для генерации случайных
чисел

public class MathGameGUI1 extends JFrame implements ActionListener {
// Объявление класса, наследующего JFrame (для GUI) и реализующего
ActionListener (для обработки событий)

    private static final int TIME_LIMIT_SECONDS_BASE = 10; //
Константа: базовое время на ответ (в секундах)
    private JLabel problemLabel; // Метка для отображения
математического примера
    private JTextField answerField; // Поле для ввода ответа
пользователя
    private JButton submitButton; // Кнопка "Ответить"
    private JLabel scoreLabel; // Метка для отображения текущего счета
    private JLabel timerLabel; // Метка для отображения оставшегося
времени
    private JButton newGameButton; // Кнопка "Начать новую игру"
    private JButton stopButton; // Кнопка "Стоп"
    private JButton changeDifficultyButton; // Кнопка "Сменить
уровень"
    private JLabel maxScoreLabel; // Метка для отображения лучшего
результата
    private JLabel messageLabel; // Метка для вывода сообщений
пользователю (правильно/неправильно/ошибка)

    private int num1; // Первое число в примере
    private int num2; // Второе число в примере
    private char operation; // Арифметическая операция ('+', '-', '*',
'/' )
    private int correctAnswer; // Правильный ответ на текущий пример
    private int score = 0; // Текущий счет игрока
    private int maxScore = 0; // Лучший достигнутый счет для текущего
уровня сложности
    private boolean gameRunning = false; // Флаг состояния игры
(запущена ли)
    private Timer timer; // Объект таймера для отсчета времени на
ответ
```

```

        private int timeLeft; // Оставшееся время (в секундах)
        private boolean timeExpired = false; // Флаг истечения времени на
ответ
        private Random random = new Random(); // Объект для генерации
случайных чисел
        private String difficulty = "Легкий"; // Текущий уровень сложности
игры
        private int maxNumber = 20; // Максимальное число для генерации
примера на текущем уровне

        public MathGameGUI1() { // Конструктор класса, инициализирующий
окно и компоненты GUI
            setTitle("Математическая Игра"); // Установка заголовка окна
            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Установка
действия при закрытии окна (завершение программы)
            setSize(700, 450); // Установка размеров окна (ширина, высота)
            setResizable(false); // Запрет изменения размера окна
            setLayout(new FlowLayout(FlowLayout.CENTER, 25, 25)); //
Установка менеджера компоновки FlowLayout с центрированием и отступами

            Font labelFont = new Font("Arial", Font.PLAIN, 28); //
Определение шрифта для меток с примером и полем ввода
            Font buttonFont = new Font("Arial", Font.BOLD, 24); //
Определение шрифта для кнопок
            Font scoreFont = new Font("Arial", Font.PLAIN, 22); //
Определение шрифта для меток счета и таймера
            Font messageFont = new Font("Arial", Font.ITALIC, 22); //
Определение шрифта для сообщений

            Color backgroundColor = new Color(240, 248, 255); //
Определение цвета фона (светло-голубой)
            Color buttonColor = new Color(173, 216, 230); // Определение
цвета обычных кнопок (светло-синий)
            Color stopButtonColor = new Color(255, 99, 71); // Определение
цвета кнопки "Стоп" (оранжево-красный)
            Color buttonTextColor = Color.BLACK; // Определение цвета
текста на обычных кнопках
            Color stopButtonTextColor = Color.WHITE; // Определение цвета
текста на кнопке "Стоп"

            getContentPane().setBackground(backgroundColor); // Установка
цвета фона для содержимого окна

            problemLabel = new JLabel("Пример: "); // Инициализация метки
примера
            problemLabel.setFont(labelFont); // Установка шрифта для метки
примера
            answerField = new JTextField(18); // Инициализация текстового
поля для ответа с шириной 18 символов
            answerField.setFont(labelFont); // Установка шрифта для поля

```

```

ответа
    submitButton = new JButton("Ответить"); // Инициализация
кнопки "Ответить"
    submitButton.addActionListener(this); // Добавление текущего
объекта как слушателя для кнопки "Ответить"
    submitButton.setFont(buttonFont); // Установка шрифта для
кнопки "Ответить"
    submitButton.setBackground(buttonColor); // Установка цвета
фона для кнопки "Ответить"
    submitButton.setForeground(buttonTextColor); // Установка
цвета текста для кнопки "Ответить"
    scoreLabel = new JLabel("Счет: 0"); // Инициализация метки
счета
    scoreLabel.setFont(scoreFont); // Установка шрифта для метки
счета
    timerLabel = new JLabel("Время: " + TIME_LIMIT_SECONDS_BASE);
// Инициализация метки таймера
    timerLabel.setFont(scoreFont); // Установка шрифта для метки
таймера
    newGameButton = new JButton("Начать новую игру"); //
Инициализация кнопки "Начать новую игру"
    newGameButton.addActionListener(e -> startNewGame()); //
Добавление лямбда-слушателя для кнопки "Начать новую игру"
    newGameButton.setFont(buttonFont); // Установка шрифта для
кнопки "Начать новую игру"
    newGameButton.setBackground(buttonColor); // Установка цвета
фона для кнопки "Начать новую игру"
    newGameButton.setForeground(buttonTextColor); // Установка
цвета текста для кнопки "Начать новую игру"
    stopButton = new JButton("Стоп"); // Инициализация кнопки
"Стоп"
    stopButton.addActionListener(e -> stopGame()); // Добавление
лямбда-слушателя для кнопки "Стоп"
    stopButton.setFont(buttonFont); // Установка шрифта для кнопки
"Стоп"
    stopButton.setBackground(stopButtonColor); // Установка цвета
фона для кнопки "Стоп"
    stopButton.setForeground(stopButtonTextColor); // Установка
цвета текста для кнопки "Стоп"
    changeDifficultyButton = new JButton("Сменить уровень"); //
Инициализация кнопки "Сменить уровень"
    changeDifficultyButton.addActionListener(e ->
chooseDifficulty()); // Добавление лямбда-слушателя для кнопки
"Сменить уровень"
    changeDifficultyButton.setFont(buttonFont); // Установка
шрифта для кнопки "Сменить уровень"
    changeDifficultyButton.setBackground(buttonColor); //
Установка цвета фона для кнопки "Сменить уровень"
    changeDifficultyButton.setForeground(buttonTextColor); //
Установка цвета текста для кнопки "Сменить уровень"

```

```

        maxScoreLabel = new JLabel("Лучший результат: 0"); //
Инициализация метки лучшего результата
        maxScoreLabel.setFont(scoreFont); // Установка шрифта для
метки лучшего результата
        messageLabel = new JLabel(""); // Инициализация метки
сообщений (изначально пустая)
        messageLabel.setFont(messageFont); // Установка шрифта для
метки сообщений
        messageLabel.setForeground(Color.RED); // Установка красного
цвета текста для сообщений

        JPanel answerPanel = new JPanel(new
FlowLayout(FlowLayout.LEFT, 15, 0)); // Создание панели для
группировки поля ввода и кнопки "Ответить"
        answerPanel.setBackground(backgroundColor); // Установка цвета
фона для панели ответа
        answerPanel.add(answerField); // Добавление поля ответа на
панель
        answerPanel.add(submitButton); // Добавление кнопки "Ответить"
на панель

        add(problemLabel); // Добавление метки примера в окно
        add(answerPanel); // Добавление панели ответа в окно
        add(scoreLabel); // Добавление метки счета в окно
        add(timerLabel); // Добавление метки таймера в окно
        add(messageLabel); // Добавление метки сообщений в окно
        add(newGameButton); // Добавление кнопки "Начать новую игру" в
окно

        add(changeDifficultyButton); // Добавление кнопки "Сменить
уровень" в окно
        add(stopButton); // Добавление кнопки "Стоп" в окно
        add(maxScoreLabel); // Добавление метки лучшего результата в
окно

        chooseDifficulty(); // Вызов метода для выбора уровня
сложности при старте
        setVisible(true); // Сделать окно видимым
    }

    private void chooseDifficulty() { // Метод для выбора уровня
сложности игры
        String[] levels = {"Легкий", "Средний", "Сложный"}; // Массив
доступных уровней сложности
        String selectedLevel = (String) JOptionPane.showInputDialog(
// Отображение диалогового окна для выбора уровня
        this, // Родительский компонент
        "Выберите уровень сложности:", // Сообщение в диалоге
        "Выбор уровня", // Заголовок диалога
        JOptionPane.PLAIN_MESSAGE, // Тип сообщения (обычное)
        null, // Иконка (нет)
        levels, // Доступные варианты выбора
        difficulty // Выбранный по умолчанию уровень
    }

```

```

        );
        if (selectedLevel != null) { // Если пользователь выбрал
уровень (не нажал "Отмена")
            difficulty = selectedLevel; // Установка выбранного уровня
сложности
            setDifficultyParameters(); // Настройка параметров игры
под выбранный уровень
            maxScore = 0; // Сброс лучшего результата при смене уровня
            maxScoreLabel.setText("Лучший результат: " + maxScore); //
Обновление метки лучшего результата
            startNewGame(); // Запуск новой игры с выбранным уровнем
        }
    }

    private void setDifficultyParameters() { // Метод для установки
параметров игры в зависимости от уровня сложности
        switch (difficulty) { // Проверка текущего уровня сложности
            case "Легкий": // Если уровень "Легкий"
                maxNumber = 15; // Максимальное число для примера - 15
                break; // Выход из switch
            case "Средний": // Если уровень "Средний"
                maxNumber = 30; // Максимальное число для примера - 30
                break; // Выход из switch
            case "Сложный": // Если уровень "Сложный"
                maxNumber = 100; // Максимальное число для примера -
100
                break; // Выход из switch
            default: // Если уровень не определен (на всякий случай)
                maxNumber = 15; // По умолчанию - 15
        }
    }

    private void generateNewProblem() { // Метод для генерации нового
математического примера
        num1 = random.nextInt(maxNumber) + 5; // Генерация первого
числа (от 5 до maxNumber+4)
        num2 = random.nextInt(maxNumber) + 3; // Генерация второго
числа (от 3 до maxNumber+2)
        if (difficulty.equals("Сложный") && random.nextInt(4) == 0) {
// Если уровень "Сложный" и случайно выбрано деление (вероятность 1/4)
            operation = '/'; // Установка операции деления
            int multiplier = random.nextInt(maxNumber / (num2 > 0 ?
num2 : 1)) + 1; // Вычисление множителя для обеспечения целого
результата деления
            num1 = num2 * multiplier; // Корректировка num1, чтобы оно
было кратно num2
            correctAnswer = multiplier; // Правильный ответ -
множитель
            problemLabel.setText("Пример: " + num1 + " " + operation +
" " + num2 + " = ?"); // Обновление текста примера

```

```

        } else { // Для других случаев или если деление не выбрано
            operation = getRandomOperation(); // Выбор случайной
операции (+, -, *)
            problemLabel.setText("Пример: " + num1 + " " + operation +
" " + num2 + " = ?"); // Обновление текста примера
            correctAnswer = calculateAnswer(num1, num2, operation); //
Вычисление правильного ответа
        }
        answerField.setText(""); // Очистка поля ввода ответа
        messageLabel.setText(""); // Очистка сообщений для
пользователя
        scoreLabel.setText("Счет: " + score); // Обновление
отображения текущего счета
        timeLeft = TIME_LIMIT_SECONDS_BASE; // Сброс оставшегося
времени до базового лимита
        timerLabel.setText("Время: " + timeLeft); // Обновление
отображения таймера
        timeExpired = false; // Сброс флага истечения времени
        startTimer(); // Запуск таймера
        answerField.requestFocus(); // Установка фокуса на поле ввода
для удобства
    }
    private char getRandomOperation() { // Метод для получения
случайной арифметической операции
        char[] operations; // Массив возможных операций
        switch (difficulty) { // Выбор операций в зависимости от
уровня сложности
            case "Легкий": // Для "Легкого" уровня
                operations = new char[]{'+', '-'}; // Только сложение
и вычитание
                break; // Выход
            case "Средний": // Для "Среднего" уровня
                operations = new char[]{'+', '-', '*'}; // Сложение,
вычитание, умножение
                break; // Выход
            case "Сложный": // Для "Сложного" уровня (деление уже
обрабатывается в generateNewProblem)
                operations = new char[]{'+', '-', '*'}; // Сложение,
вычитание, умножение
                break; // Выход
            default: // По умолчанию
                operations = new char[]{'+', '-'}; // Сложение и
вычитание
        }
        return operations[random.nextInt(operations.length)]; //
Возврат случайной операции из массива
    }

    private int calculateAnswer(int num1, int num2, char operation) {
// Метод для вычисления ответа на математический пример

```

```

        return switch (operation) { // Использование switch-выражения
для операций
            case '+' -> num1 + num2; // Сложение
            case '-' -> num1 - num2; // Вычитание
            case '*' -> num1 * num2; // Умножение
            default -> 0; // По умолчанию (в случае неизвестной
операции)
        };
    }

    private void startTimer() { // Метод для запуска таймера
        stopTimer(); // Остановка любого активного таймера перед
запуском нового
        timer = new Timer(1000, new ActionListener() { // Создание
нового объекта Timer, срабатывающего каждую 1000 мс (1 секунда)
            @Override // Переопределение метода actionPerformed
интерфейса ActionListener
            public void actionPerformed(ActionEvent e) { // Действия,
выполняемые каждую секунду
                if (gameRunning) { // Если игра активна
                    timeLeft--; // Уменьшение оставшегося времени
                    timerLabel.setText("Время: " + timeLeft); //
Обновление текста метки таймера
                    if (timeLeft <= 0) { // Если время истекло
                        stopTimer(); // Остановка таймера
                        messageLabel.setText("Время вышло! Правильный
ответ: " + correctAnswer); // Сообщение об истечении времени
                        scoreLabel.setText("Счет: " + score); //
Обновление метки счета (отображает текущий счет при завершении)
                        gameRunning = false; // Игра остановлена
                        timeExpired = true; // Установка флага
истечения времени
                        updateMaxScore(); // Обновление лучшего
результата
                    }
                }
            }
        });
        timer.start(); // Запуск таймера
    }

    private void stopTimer() { // Метод для остановки таймера
        if (timer != null && timer.isRunning()) { // Если таймер
существует и активен
            timer.stop(); // Остановка таймера
        }
    }

    private void stopGame() { // Метод для принудительной остановки
игры
        gameRunning = false; // Установка флага, что игра не активна

```

```

        stopTimer(); // Остановка таймера
        messageLabel.setText("Игра остановлена. Итоговый счет: " +
score); // Вывод сообщения об остановке и итоговом счете
    }
    private void checkAnswer() { // Метод для проверки ответа
пользователя
        if (!gameRunning) { // Если игра не активна (например, уже
завершилась)
            return; // Выход из метода
        }
        stopTimer(); // Остановка таймера на время проверки ответа
        String userAnswerStr = answerField.getText().trim(); //
Получение текста из поля ответа и удаление пробелов

        if (timeExpired) { // Если флаг timeExpired установлен (время
вышло)
            messageLabel.setText("Время вышло! Правильный ответ: " +
correctAnswer); // Сообщение об истечении времени
            gameRunning = false; // Игра останавливается
            updateMaxScore(); // Обновление лучшего результата
            return; // Выход из метода
        }

        if (!userAnswerStr.isEmpty()) { // Если пользователь ввел что-
то в поле ответа
            try {
                int userAnswer = Integer.parseInt(userAnswerStr); //
Попытка преобразовать ввод в целое число
                if (userAnswer == correctAnswer) { // Если ответ
правильный
                    score++; // Увеличение счета
                    scoreLabel.setText("Правильно! Ваш счет: " +
score); // Обновление метки счета
                    gameRunning = true; // Игра продолжается
                    generateNewProblem(); // Генерация нового примера
                } else { // Если ответ неправильный
                    messageLabel.setText("Неправильно! Правильный
ответ: " + correctAnswer); // Сообщение о неправильном ответе
                    gameRunning = false; // Игра останавливается
                    updateMaxScore(); // Обновление лучшего результата
                }
            } catch (NumberFormatException e) { // Если ввод не
является числом (ошибка парсинга)
                messageLabel.setText("Некорректный ввод. Попробуйте
ввести число."); // Сообщение о некорректном вводе
                startTimer(); // Перезапуск таймера, так как ответ не
был валидирован как число
            }
        } else { // Если поле ответа пустое
            messageLabel.setText("Пожалуйста, введите ответ."); //

```

```

        Просьба ввести ответ
            startTimer(); // Перезапуск таймера
        }
    }

    private void startNewGame() { // Метод для запуска новой игры
        score = 0; // Сброс текущего счета
        scoreLabel.setText("Счет: 0"); // Обновление метки счета
        gameRunning = true; // Установка флага, что игра активна
        generateNewProblem(); // Генерация первого примера новой игры
        maxScoreLabel.setText("Лучший результат: " + maxScore); //
        Обновление метки лучшего результата (сброс не происходит, отображается
        предыдущий рекорд)
    }

    private void updateMaxScore() { // Метод для обновления лучшего
    результата
        if (score > maxScore) { // Если текущий счет превышает лучший
            maxScore = score; // Обновление лучшего результата
            maxScoreLabel.setText("Новый лучший результат: " +
            maxScore); // Обновление метки с сообщением о новом рекорде
        }
    }

    public static void main(String[] args) { // Основной метод, точка
    входа в программу
        SwingUtilities.invokeLater(MathGameGUI::new); // Запуск GUI в
        потоке диспетчеризации событий Swing для обеспечения безопасности
        потоков
    }

    @Override // Аннотация, указывающая, что метод переопределяет
    метод из интерфейса или родительского класса
    public void actionPerformed(ActionEvent e) { // Метод-обработчик
    событий (для кнопок, у которых this добавлен как слушатель)
        if (e.getSource() == submitButton) { // Если событие произошло
        от кнопки "Ответить"
            checkAnswer(); // Вызов метода проверки ответа
        } else if (e.getSource() == newGameButton) { // Если событие
        произошло от кнопки "Начать новую игру"
            startNewGame(); // Вызов метода запуска новой игры
        } else if (e.getSource() == changeDifficultyButton) { // Если
        событие произошло от кнопки "Сменить уровень"
            chooseDifficulty(); // Вызов метода выбора уровня
        сложности
        }
    }
}

```

